Reporte

Nombre:	**Matrícula**:
Ximena Montserrat Ovalle Vallejo	1999238
Santiago Almanza Moreno	2001903
Javier Hernández Huerta	2089204

Nombre del curso:	**Nombre del profesor**:
Componentes de Lógica de Negocio	Miguel Ángel Gómez Marroquín
Módulo Netacad:	**Actividad**: Actividad Unidad 5. **Laboratorio guiado**

Fecha: 03/ 11/ 2026

Bibliografia (Formato APA)

- Astral. (s. f.). Ruff documentation. https://docs.astral.sh/ruff/
- Batchelder, N. (s. f.). Coverage.py documentation. https://coverage.readthedocs.io/
- Beck, K. (2002). Test-driven development: By example. Addison-Wesley.
- GitHub. (s. f.). GitHub Actions documentation. https://docs.github.com/actions
- pytest-dev. (s. f.). pytest documentation. https://docs.pytest.org/
- Python Software Foundation. (s. f.). unittest.mock: Mock object library. https://docs.python.org/3/library/unittest.mock.html
- SQLAlchemy. (s. f.). SQLAlchemy 2.1 documentation. https://docs.sqlalchemy.org/

Objetivo:

Comprobar con pruebas automatizadas que las reglas de negocio se cumplen, que la persistencia y los servicios externos se comportan como se espera y que el pipeline de CI rechaza cambios que rompen las pruebas; primero sobre el proyecto base de pedidos y después sobre las reglas reales de nuestra empresa del PIA, RutaNorte Logistics.

REPOSITORIO GITHUB:

https://github.com/ximenaovallev/CLN_AU5_Equipo2

Procedimiento:

Se partió del ZIP del laboratorio (proyecto base, CSV y base SQLite didáctica) y del código de la Unidad 3 de RutaNorte (30 pruebas). Se siguió la guía paso por paso y cada fase quedó documentada con capturas (incluida la fase RED del ciclo TDD). Todas las salidas de terminal, matrices y reportes de cobertura se generaron ejecutando el código; las capturas de terminal (E1 a E7 y E9) se tomaron en la laptop de una integrante del equipo (Windows, Python 3.14.7) con las mismas versiones fijadas en requirements.txt; el pipeline de E8 se ejecutó en GitHub Actions (Ubuntu, Python 3.11). Como apoyo se conservan algunas capturas de la verificación previa en Linux (simulación local del pipeline, prueba de mutación, hallazgo de Ruff y reporte HTML de cobertura del PIA y su línea 154).

Contenido

1. Resumen de resultados
2. Estrategia de pruebas: unitaria, integración y E2E
3. E1 · Paso 1: instalar y comprobar el proyecto
4. E2 · Paso 2: prueba unitaria y Arrange–Act–Assert
5. E3 · Paso 3: TDD real (Red → Green → Refactor)
6. E4 · Paso 4: integración con base de datos y transacciones
7. E5 · Paso 5: servicios externos sin cobros reales
8. E6 · Paso 6: seguridad y autorización
9. E7 · Paso 7: cobertura y calidad
10. E8 · Paso 8: GitHub Actions
11. E9 · Paso 9: aplicación a RutaNorte Logistics (PIA)
12. Vibe Coding con IA: uso responsable
13. Lista de entrega y autoevaluación
14. Reflexión final
15. Conclusiones

1. Resumen de resultados

Parte	Pruebas	Cobertura	Ruff	Matriz de casos
Laboratorio (pedidos)	51 PASSED + 1 XFAIL documentado	100 % de src/	Sin errores	CP01–CP15: 15/15 PASA
PIA RutaNorte Logistics	56 PASSED (30 U3 + 26 U5)	99.76 % de rutanorte (ramas incluidas)	Sin errores	PIA-01–PIA-23: 23/23 PASA
Pipeline CI (2 jobs)	verde → rojo → verde	umbral --cov-fail-under=80	paso de lint	el job PIA falla si un caso no pasa

1.1 Resultados de aprendizaje y dónde se evidencian

Resultado de aprendizaje	Evidencia en este trabajo
Distinguir pruebas unitarias, integración y E2E	E2 (unitarias), E4–E5 (integración), sección 2 y E9 (marcadores unitaria / integracion / flujo; estrategia E2E documentada sin fingir una interfaz).
Aplicar Arrange–Act–Assert y el ciclo TDD Red–Green–Refactor	E2 (pruebas con comentarios AAA) y E3 (Figuras 3, 4 y 5: RED → GREEN → REFACTOR).
Verificar persistencia y rollback de SQLAlchemy sin modificar datos de producción	E4 (SQLite en memoria por prueba; lectura desde sesión nueva; unicidad) y PIA-16 a PIA-19 (SqlAlchemyUnitOfWork).
Simular cobros con mocks, probar autorización y revisar cobertura	E5 (pasarela con Mock(spec)), E6 (matriz A01–A09), E7 y E9 (reportes HTML), PIA-20 a 22 (bus de eventos simulado), PIA-13 a 15 (autoridad de cancelación).
Interpretar un pipeline CI y entregar evidencia reproducible	E8 (ci.yml con 2 jobs; GitHub Actions #2 verde, #3 roja y #4 verde), versiones fijadas, scripts/ci_local.sh y matrices generadas por script.

2. Estrategia de pruebas: unitaria, integración y E2E

Seguimos la pirámide 70/20/10 como referencia: muchas pruebas rápidas del dominio aislado, menos pruebas que tocan infraestructura y muy pocas de recorrido completo. La diferencia no es el tamaño de la prueba sino **qué dependencias reales intervienen**.

Nivel	Qué verifica	Ejemplos	Dependencias
Unitaria	Una regla del dominio aislada	test_vip_200_recibe_15_por_ciento_y_cobra_170; PIA-06 (carga igual a la capacidad)	Ninguna: sin BD, red ni pasarela
Integración	Dominio + persistencia o servicio externo	test_commit_pedido_250_id_2 …; PIA-17 (rollback con SQLAlchemy); PIA-20 (bus simulado)	SQLite en memoria; unittest.mock
Flujo (sustituto documentado de E2E)	Proceso completo CU-01→CU-05 por la Service Layer	PIA-23	Implementaciones en memoria
E2E (propuesto)	Usuario real → API/UI → BD	No se implementa: no existe API ni interfaz	Navegador, API, BD desechable

Escenario E2E propuesto para RutaNorte (cuando exista la API): el cliente acepta la cotización por HTTP → Operaciones asigna unidad y operador → el operador registra llegada y entrega con fotografía desde el móvil → Atención al Cliente abre el servicio en el navegador y ve el estado “Entregado” con su evidencia. Se automatizaría con Playwright contra un entorno de staging con una base PostgreSQL desechable. No se presenta como hecho porque la guía pide no fingir una interfaz inexistente.

Conteo de las pruebas U5 del PIA por nivel (pytest -m): unitaria 10, integración 13, flujo 1 (Figura 25).

2.1 Plan de pruebas

Antes de escribir las pruebas se definió qué funcionalidades probar, con qué regla y cuál es el resultado esperado. Las matrices de casos (evidencias/matriz_resultados_laboratorio.csv con CP01–CP15 y evidencias/matriz_casos_pia.csv con PIA-01 a PIA-23) detallan cada caso con su resultado real. La matriz del laboratorio cumple la distribución del manual: 5 casos de éxito, 4 de error, 3 de límite y 3 de integración.

Funcionalidad a probar	Resultado esperado	Nivel	Casos y evidencia
Confirmar pedido (Order.confirm)	VIP con 15 % de descuento; total ≤ 0 o cliente con deuda → error; un pedido confirmado no se confirma otra vez	Unitaria	CP01–CP12 (5 éxito, 4 error, 3 límite); E2 y E3
Persistencia del pedido (SqlOrderRepository)	Visible tras commit; ausente tras rollback; id duplicado rechazado sin alterar el original	Integración	CP14, CP15; E4
Cobro con pasarela (OrderService)	Pago aprobado devuelve el importe; rechazado lanza RuntimeError; sin cargos reales	Integración (mock)	CP13; E5
Cancelación de pedidos (can_cancel)	SHIPPED nunca se cancela; el administrador sí puede; el dueño solo antes de 24 h	Unitaria	A01–A09; E6
R1 · Generar orden de servicio	Solo desde una cotización aceptada	Unitaria	PIA-01 a PIA-03; E9
R2 · Asignar unidad y operador	Capacidad suficiente (igual a la capacidad se permite) y operador disponible	Unitaria	PIA-04 a PIA-08; E9
R3 · Registrar entrega	Requiere llegada a destino, receptor y evidencia; una incidencia no entrega	Unitaria / integración	PIA-09 a PIA-12; E9
R4 · Cancelar servicio (S-04)	Solo roles autorizados; nunca un servicio entregado	Integración	PIA-13 a PIA-15; E9
Persistencia RutaNorte (Unit of Work)	Commit y rollback reales; misma conducta en memoria y con SQLAlchemy	Integración	PIA-16 a PIA-19; E9
Notificación a Operaciones (bus de eventos)	Publica solo después de confirmar; un error de negocio no publica	Integración (mock)	PIA-20 a PIA-22; E9

Proceso completo CU-01 → CU-05	Atención al Cliente ve ENTREGADO con sus eventos; cumplimiento 1.0	Flujo	PIA-23; E9
Calidad continua (CI)	Ruff, pruebas, cobertura ≥ 80 % y build en cada push; rojo si algo falla	Pipeline	E7 y E8

3. E1 · Paso 1: instalar y comprobar el proyecto

Se creó el entorno virtual en Windows (Python 3.14.7; el pipeline de GitHub usa Python 3.11), se instalaron las dependencias y se ejecutó la suite completa del laboratorio: **51 pruebas PASSED y 1 XFAIL documentado** (las 13 pruebas del proyecto base más las agregadas en los Pasos 2 a 7).

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pytest -v
```

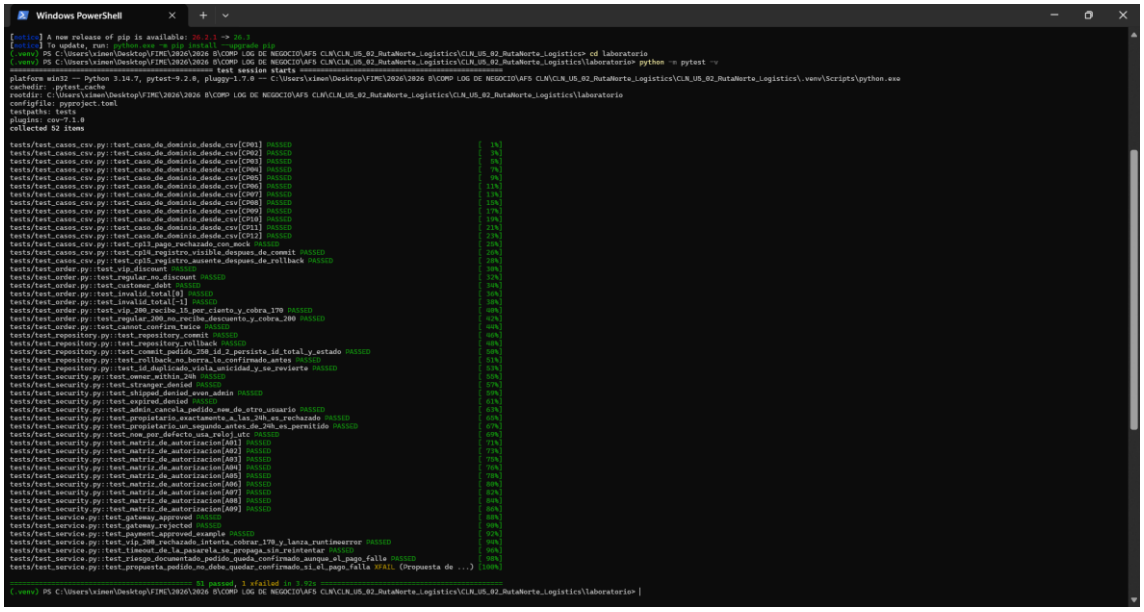


Figura 1. python -m pytest -v en Windows (Python 3.14.7): 51 passed, 1 xfailed.

¿Para qué sirven pytest y el entorno virtual? (3 líneas)

El entorno virtual (.venv) aísla las bibliotecas del curso para que el proyecto use exactamente las versiones de requirements.txt y no las del sistema.

pytest descubre las funciones test_*, las ejecuta y compara cada assert contra el resultado real, reportando PASSED o FAILED con el detalle de la diferencia.

Juntos hacen la verificación reproducible: cualquier integrante o el CI obtiene el mismo resultado con los mismos comandos.

4. E2 · Paso 2: prueba unitaria y Arrange–Act–Assert

Order.confirm() rechaza totales ≤ 0, rechaza clientes con deuda, aplica 15 % de descuento VIP y cambia el estado a CONFIRMED. Las dos pruebas nuevas no usan base de datos ni pasarela: solo el objeto de dominio.

```
def test_vip_200_recibe_15_por_ciento_y_cobra_170():
        # Arrange
        order = Order(200, vip=True)
        # Act
        amount = order.confirm()
        # Assert
        assert amount == 170
        assert order.discount == 30
        assert order.status == "CONFIRMED"

def test_regular_200_no_recibe_descuento_y_cobra_200():
        order = Order(200, vip=False)          # Arrange
        amount = order.confirm()               # Act
        assert amount == 200                   # Assert
        assert order.discount == 0.0 and order.status == "CONFIRMED"
```

Entrada	Regla	Resultado esperado	Resultado real	Estado
total=200, vip=True	VIP: 15 % de descuento; cobro = total − descuento	170 (descuento 30)	170.0 (descuento 30.0), CONFIRMED	PASSED
total=200, vip=False	Regular: sin descuento	200 (descuento 0)	200.0 (descuento 0.0), CONFIRMED	PASSED

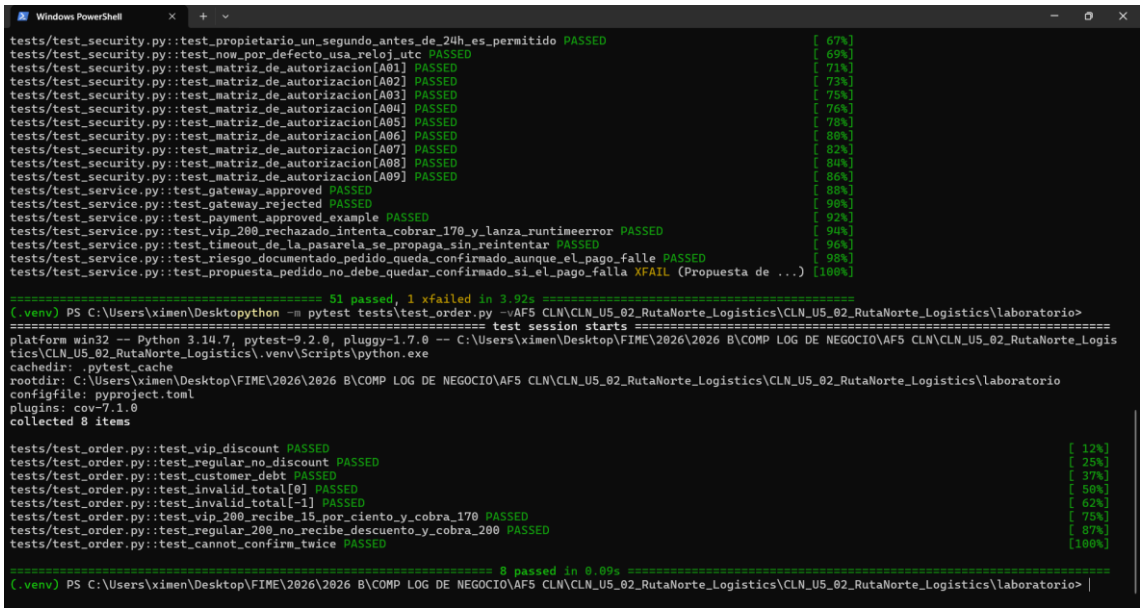


Figura 2. python -m pytest tests/test_order.py -v: las dos pruebas nuevas en verde.

5. E3 · Paso 3: TDD real (Red → Green → Refactor)

Regla nueva: **no confirmar un pedido que ya fue confirmado** (caso CP08 de casos_de_prueba.csv). No se usó la regla de total positivo porque ya existía y no habría producido la fase RED.

RED · la prueba se escribe antes del código

```
def test_cannot_confirm_twice():
        order = Order(100)
        order.confirm()
        with pytest.raises(ValueError, match="already confirmed"):
        order.confirm()
```

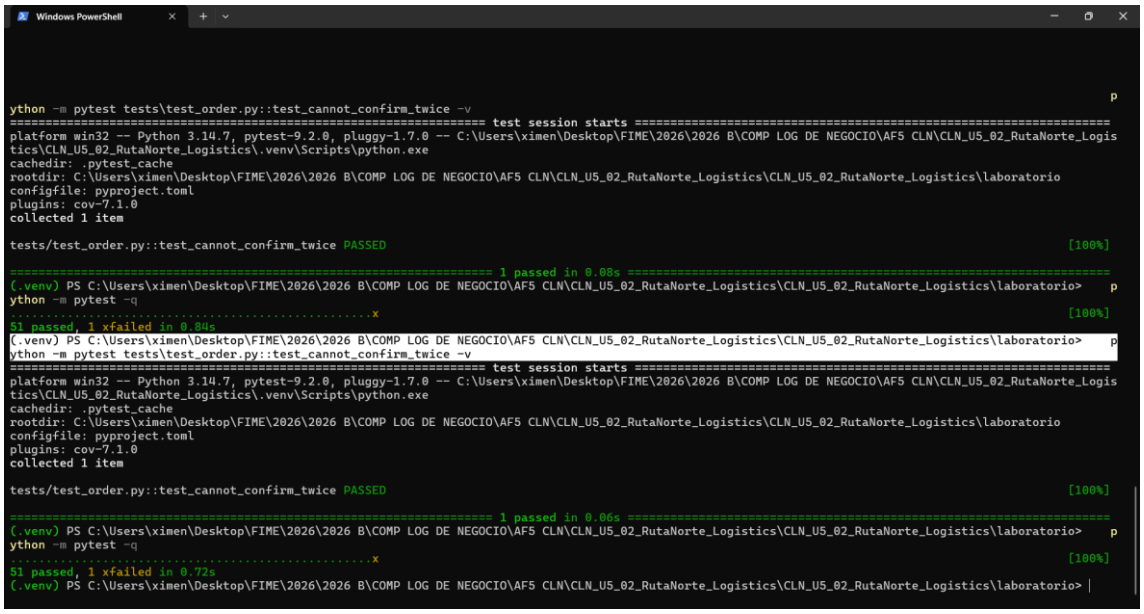


Figura 3. Fase RED: DID NOT RAISE ValueError, porque el dominio permitía confirmar dos veces.

GREEN · implementación mínima

```
def confirm(self, customer_debt=0):
        if self.status == "CONFIRMED":
        raise ValueError("already confirmed")
        if self.total <= 0:
        ...
```

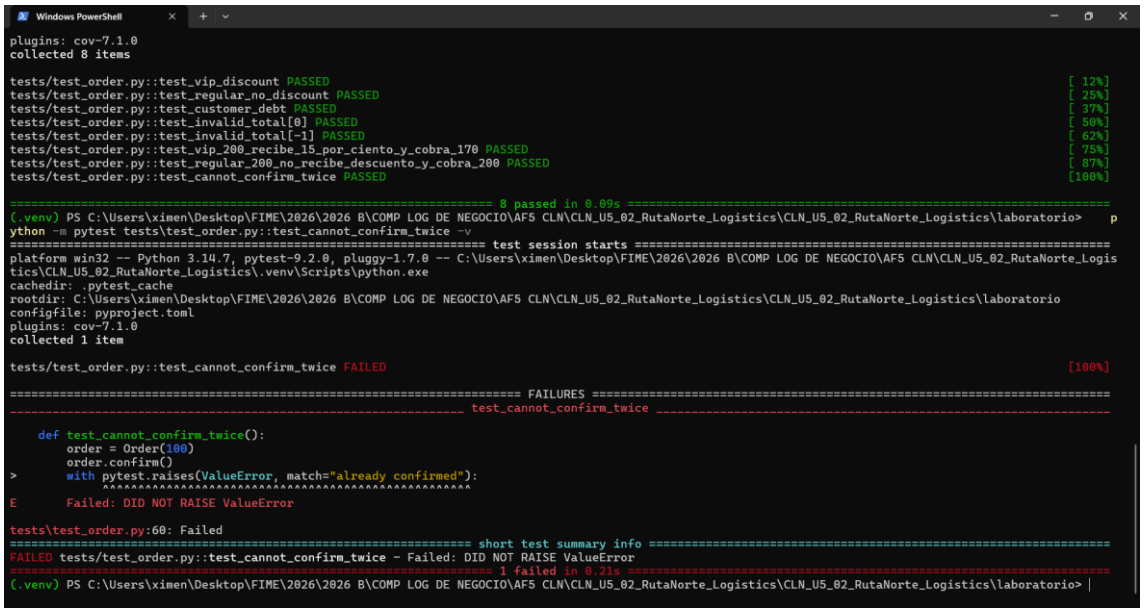


Figura 4. Fase GREEN: la prueba pasa y la suite completa sigue en verde (51 passed, 1 xfailed).

REFACTOR · mejorar sin cambiar la conducta

Las tres validaciones se extrajeron a un método privado y los literales a constantes de dominio. La interfaz pública (confirm, CustomerHasDebt, mensajes de error) no cambió y toda la suite se volvió a ejecutar.

```
VIP_DISCOUNT_RATE = 0.15
CONFIRMED = "CONFIRMED"

def confirm(self, customer_debt=0):
    self._ensure_can_be_confirmed(customer_debt)
        self.discount = round(self.total * VIP_DISCOUNT_RATE, 2) if self.vip else 0.0
        self.status = CONFIRMED
        return self.total - self.discount

def _ensure_can_be_confirmed(self, customer_debt):
        if self.status == CONFIRMED:
        raise ValueError("already confirmed")
        if self.total <= 0:
        raise ValueError("total must be positive")
        if customer_debt > 0:
        raise CustomerHasDebt("outstanding debt")
```

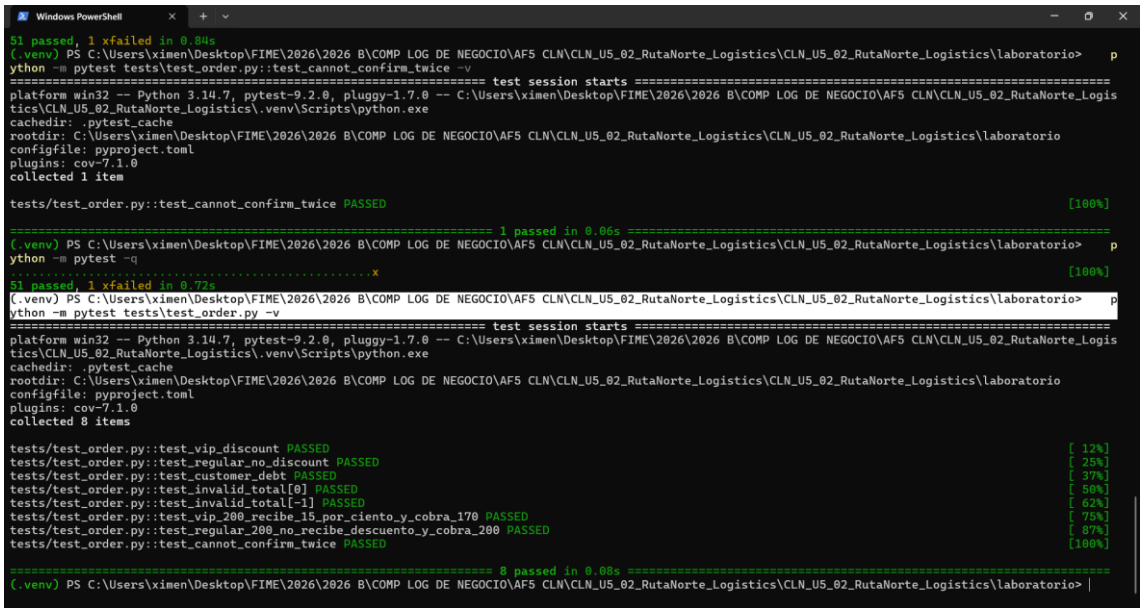


Figura 5. Fase REFACTOR: misma conducta; tests/test_order.py en verde (8 passed).

¿Por qué TDD no es simplemente ejecutar pruebas después de programar?

En TDD la prueba es la especificación de la regla y se escribe primero; verla fallar (RED) demuestra que la prueba sí es capaz de detectar la ausencia de la regla. Una prueba escrita después puede pasar por accidente (por ejemplo, porque no verifica nada útil) y nunca sabremos si detecta el defecto. Además, el código se limita a lo mínimo para pasar (GREEN) y el diseño se mejora con la red de seguridad de la suite (REFACTOR). Por eso la evidencia incluye la captura en rojo (Figura 3), no solo capturas verdes (Figuras 4 y 5).

Consecuencia en los datos: la fila CP08 del CSV esperaba “CONFIRMED (comportamiento actual; ampliar con TDD)”. Después del ciclo, la expectativa se actualizó a ValueError: already confirmed, y así aparece en la matriz de resultados.

6. E4 · Paso 4: integración con base de datos y transacciones

Aquí sí interviene SQLAlchemy. Cada prueba llama a make_engine(), que crea una base **SQLite en memoria** nueva y vacía: no se toca ninguna base de producción ni el archivo didáctico pedidos_prueba.sqlite3.

```
def test_commit_pedido_250_id_2_persiste_id_total_y_estado():
        engine = make_engine()                  # Arrange: base aislada
        order = Order(250); order.confirm()
        with Session(engine) as session:                # Act
       SqlOrderRepository(session).save(2, order)
        session.commit()
        with Session(engine) as other_session:       # Assert desde una sesión
NUEVA
```

```
        row = other_session.get(OrderRow, 2)
        assert (row.id, row.total, row.status) == (2, 250, "CONFIRMED")
```

Se agregaron dos pruebas más que la guía y el manual sugieren: **un rollback no borra lo confirmado antes** (commit del pedido 1, rollback del pedido 2 → solo existe el 1) y **la restricción de unicidad** (guardar dos veces id=1 produce IntegrityError y el total original 120 no cambia).

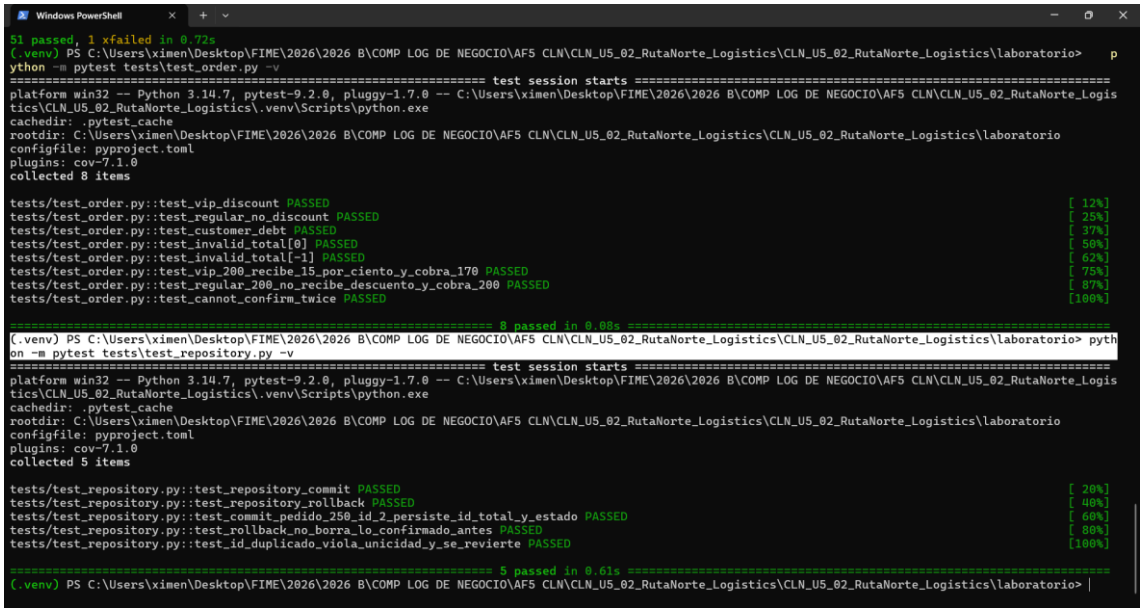


Figura 6. python -m pytest tests/test_repository.py -v: 5 pruebas de integración en verde.

Diferencia entre commit y rollback

Mientras una transacción está abierta, los cambios agregados a la sesión son provisionales. **commit()** cierra la transacción y los vuelve permanentes en la base: otra sesión puede leerlos, y por eso la prueba consulta desde una sesión nueva y no desde la caché de la anterior. **rollback()** descarta todo lo pendiente desde el último commit y deja la base como estaba; no afecta lo que ya se había confirmado. En el ejemplo, la sesión es el límite transaccional: o se guarda el pedido completo o no se guarda nada. SQLite en memoria es rápido y aislado, pero no sustituye las pruebas contra el motor productivo (por ejemplo PostgreSQL).

7. E5 · Paso 5: servicios externos sin cobros reales

La pasarela se sustituye por un Mock con la interfaz del puerto (spec=PaymentGateway): si el código llamara un método que la pasarela no tiene, la prueba fallaría. No se abre ninguna conexión ni se mueve dinero.

```
def test_vip_200_rechazado_intenta_cobrar_170_y_lanza_runtimeerror():
        gateway = Mock(spec=PaymentGateway)
        gateway.charge.return_value = False
        order = Order(200, vip=True)
        with pytest.raises(RuntimeError, match="payment rejected"):
       OrderService(gateway).checkout(order)
   gateway.charge.assert_called_once_with(170)
        assert gateway.mock_calls == [("charge", (170,), {})]   # una sola llamada
simulada
```

Ruta	Respuesta del mock	Resultado verificado
Aprobado	charge → True	devuelve 100; charge llamado una vez con 100
Rechazado (VIP $200)	charge → False	intento de cobro por 170; RuntimeError("payment rejected")
Timeout (adicional, manual Tema 4)	charge lanza TimeoutError	se propaga sin reintentos (1 llamada)

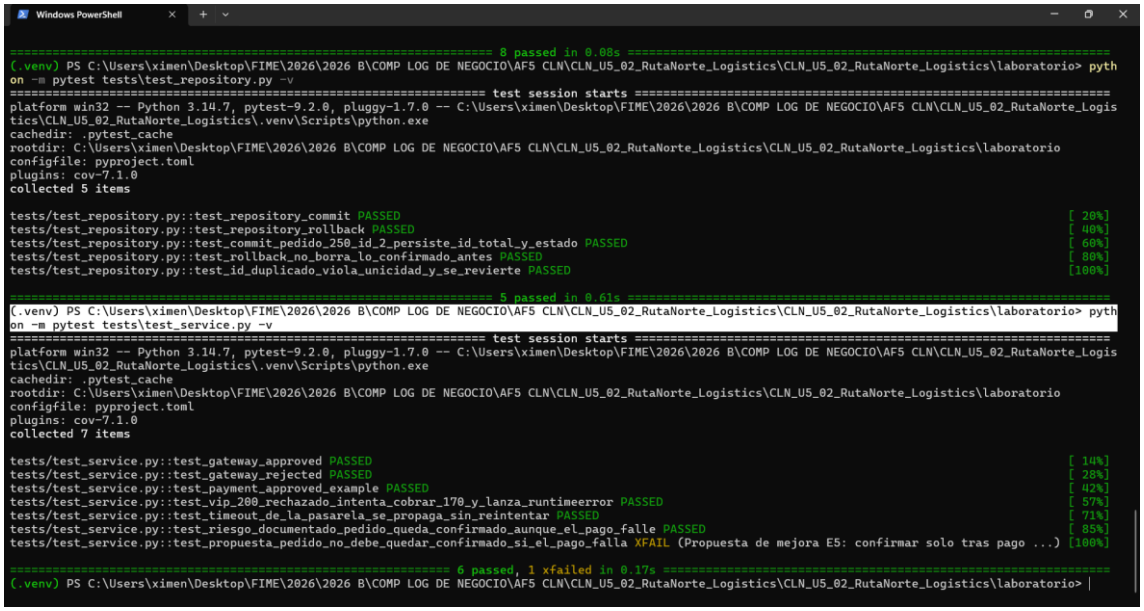


Figura 7. python -m pytest tests/test_service.py -v: rutas aprobada, rechazada y timeout; el XFAIL es la mejora propuesta.

Riesgo de consistencia y propuesta de mejora

checkout() llama a order.confirm() **antes** de saber si el cobro fue aceptado. Si la pasarela rechaza, el pedido queda en CONFIRMED sin pago. No se oculta: test_riesgo_documentado_pedido_queda_confirmado_aunque_el_pago_falle demuestra el comportamiento actual, y test_propuesta_pedido_no_debe_quedar_confirmado_si_el_pago_falla está marcado xfail(strict=True) con la conducta deseada (si alguien corrige el código, el xfail estricto avisará para convertirla en prueba normal).

Propuesta: separar el cálculo del importe de la confirmación: (1) validar y calcular el cobro sin cambiar el estado; (2) cobrar; (3) solo si charge() devuelve True, cambiar a CONFIRMED y guardar en la misma transacción. Si el cobro falla, el pedido queda en NEW o pasa a un estado explícito PAYMENT_FAILED. Si el cobro es asíncrono, usar un estado intermedio PENDING_PAYMENT y una compensación.

8. E6 · Paso 6: seguridad y autorización

Reglas implementadas en can_cancel: un pedido SHIPPED nunca se cancela, **ni siquiera por un administrador**; en otros estados el administrador puede cancelar; el propietario no administrador solo durante las primeras 24 horas (estrictamente menos de 24 h). Se probó lo que el proyecto implementa, no la regla del slide original donde el administrador siempre puede cancelar.

Todas las pruebas nuevas usan una fecha fija (NOW = 2026-11-02 12:00 UTC) y pasan now explícitamente, así que el resultado no depende del reloj.

```
NOW = datetime(2026, 11, 2, 12, 0, tzinfo=UTC)

def test_admin_cancela_pedido_new_de_otro_usuario():
        assert can_cancel("NEW", OWNER, ADMIN, True, NOW -
timedelta(hours=72), NOW)

def test_propietario_exactamente_a_las_24h_es_rechazado():
   assert not can_cancel("NEW", OWNER, OWNER, False, NOW -
timedelta(hours=24), NOW)
```

Caso	Rol	¿Propietario?	Estado	Antigüedad	Resultado
A01	Propietario	Sí	NEW	1 h	Permitido
A02	Propietario	Sí	NEW	24 h	Denegado (24 h exactas: el límite es < 24 h)
A03	Propietario	Sí	NEW	25 h	Denegado (vencido)
A04	Tercero (no admin)	No	NEW	1 h	Denegado
A05	Administrador	No	NEW	72 h	Permitido (admin, sin límite de tiempo)
A06	Propietario	Sí	CONFIRMED	2 h	Permitido
A07	Administrador	No	CONFIRMED	2 h	Permitido
A08	Propietario	Sí	SHIPPED	1 h	Denegado
A09	Administrador	No	SHIPPED	1 h	Denegado (enviado prevalece incluso para admin)

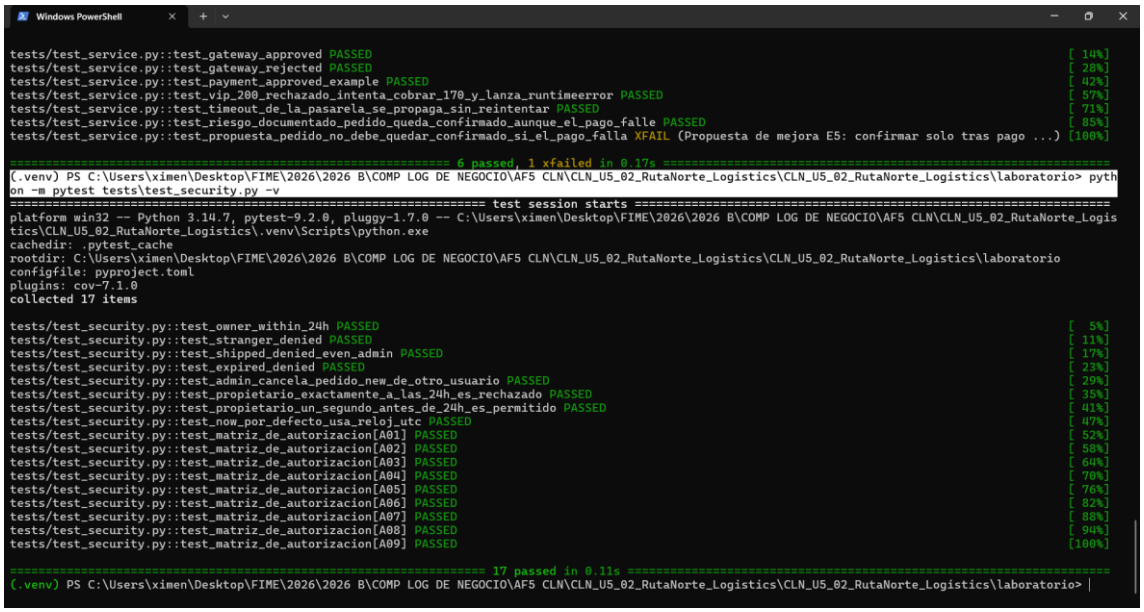


Figura 8. python -m pytest tests/test_security.py -v: 17 pruebas, incluida la matriz A01–A09 parametrizada.

9. E7 · Paso 7: cobertura y calidad

```
python -m ruff check src tests
python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80
```

Resultado: Ruff sin errores (Figura 11) y **100 % de cobertura** de src/ (Figuras 9 y 10). La laptop cuenta 56 sentencias con Python 3.14 y GitHub Actions 57 con Python 3.11 (Figura 19); la diferencia es solo cómo cada versión cuenta las líneas. La cobertura del 80 % o más significa que al menos esa proporción de líneas se ejecutó durante las pruebas; **no** significa que se detectó ese porcentaje de defectos.

Advertencias ResourceWarning: en la laptop (Python 3.14) pytest muestra 5 avisos “ResourceWarning: unclosed database in sqlite3.Connection”. No son fallos: todas las pruebas pasan y la cobertura no cambia. Python 3.13 y posteriores avisan cuando una conexión SQLite en memoria se libera al terminar la prueba sin cerrarse explícitamente; en GitHub Actions (Python 3.11) no aparecen. Mejora posible: cerrar el motor con engine.dispose() al final de cada prueba de repositorio.

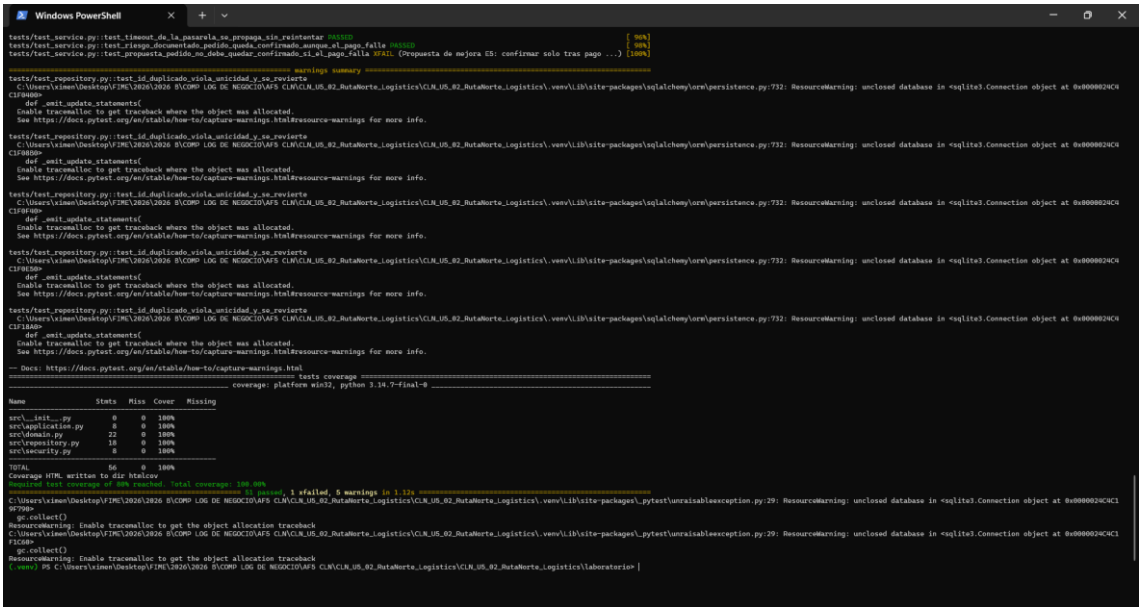


Figura 9. Cobertura en terminal: 100 % de src/ (51 passed, 1 xfailed).

Coverage report: 100%

Files | Functions | Classes

filter...

hide covered

coverage.py v7.17.0, created at 2026-11-02 16:28 -0600

File ▲	statements	missing	excluded	coverage
src__init__.py	0	0	0	100%
src\application.py	8	0	0	100%
src\domain.py	22	0	0	100%
src\repository.py	18	0	0	100%
src\security.py	8	0	0	100%
Total	**56**	**0**	**0**	**100%**

coverage.py v7.17.0, created at 2026-11-02 16:28 -0600

Figura 10. Reporte HTML (htmlcov/index.html) del laboratorio.

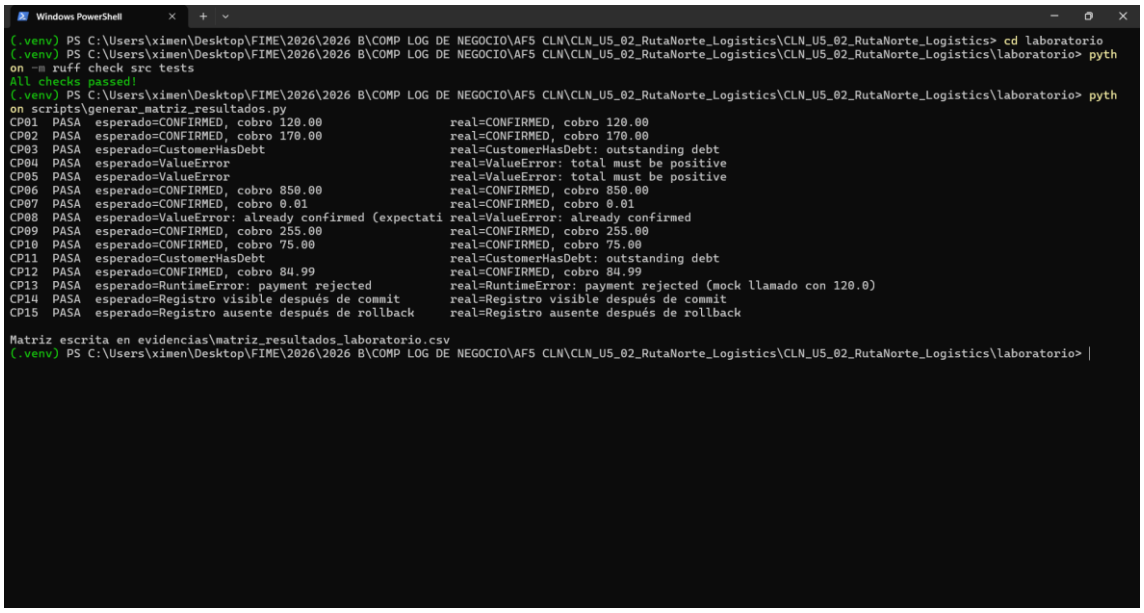


Figura 11. Ruff sin errores y matriz CP01–CP15 en PASA (python scripts/generar_matriz_resultados.py).

Hallazgo de calidad: el lint dependía de la fecha de instalación requirements.txt pedía ruff>=0.6 sin tope. La versión que se instala hoy (0.17.3) activa reglas nuevas (orden de imports I001 y alias datetime.UTC UP017) y el **ZIP base original, sin tocarlo, falla el lint con 12 errores**: el pipeline habría quedado en rojo sin que nadie cambiara el código. Se corrigió con ruff --fix (solo formato, sin cambiar conducta) y se fijaron las versiones (pytest 9.2.0, pytest-cov 7.1.0, SQLAlchemy 2.1.6, ruff 0.17.3) para que el CI sea reproducible.

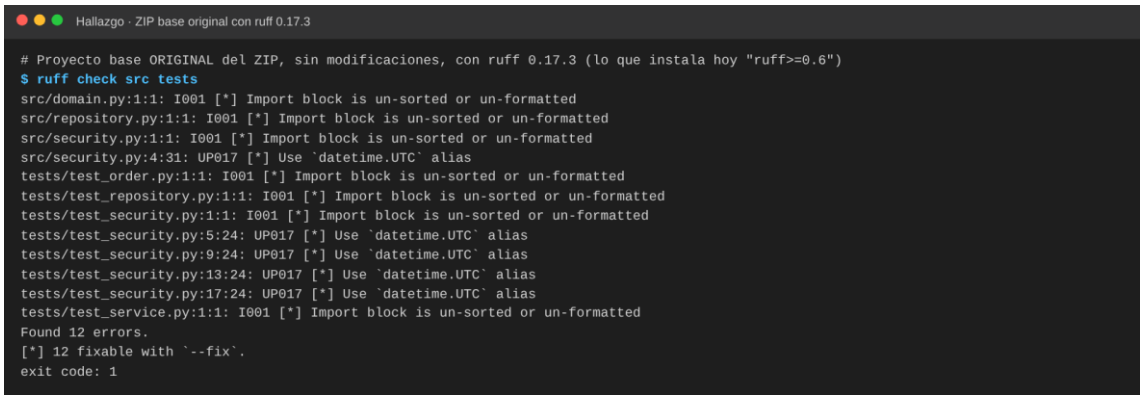


Figura 12. El proyecto base original falla el lint con la versión actual de Ruff.

Una rama o escenario aún no probado

Con 100 % de cobertura todavía hay comportamiento sin especificar: **customer_debt negativo**. La condición customer_debt > 0 trata un saldo a favor (por ejemplo −50) igual que “sin deuda”, y ninguna prueba dice si eso es correcto, porque la regla no está definida. Otro ejemplo: si la pasarela devolviera None en lugar de True/False, checkout lo trataría como rechazo. La cobertura dice qué líneas se ejecutaron, no si todas las entradas relevantes tienen una expectativa; por eso no se agregaron pruebas artificiales solo para subir el número.

10. E8 · Paso 8: automatizar la revisión con GitHub Actions

GitHub solo lee workflows en .github/workflows de la **raíz** del repositorio, así que el ci.yml del ZIP se movió a la raíz y se dividió en dos jobs independientes con los mismos pasos originales (checkout, Python 3.11, dependencias, Ruff, pytest con --cov-fail-under=80, compileall). El job del PIA agrega la generación de la matriz de casos, que termina con error si algún caso no pasa. Ambos publican el reporte HTML como artefacto. El YAML se validó con actionlint.

```
name: CI Business Logic
on: [push, pull_request]
jobs:
  laboratorio:            # working-directory: laboratorio
      steps: checkout → setup-python 3.11 → pip install → ruff → pytest --cov=src ... →
compileall → artefacto
  pia:                    # working-directory: pia
      steps: checkout → setup-python 3.11 → pip install → ruff → pytest --
cov=rutanorte ... → matriz PIA → compileall → artefacto
```

Comprobación del bloqueo. Antes de publicar, se reprodujeron localmente los mismos pasos del workflow con scripts/ci_local.sh, incluida una aserción falsa temporal (test_vip_discount espera 90 en lugar de 85) y su reversión (Figuras 13 a 15). Después se repitió la secuencia en GitHub Actions (Figuras 16 a 20).

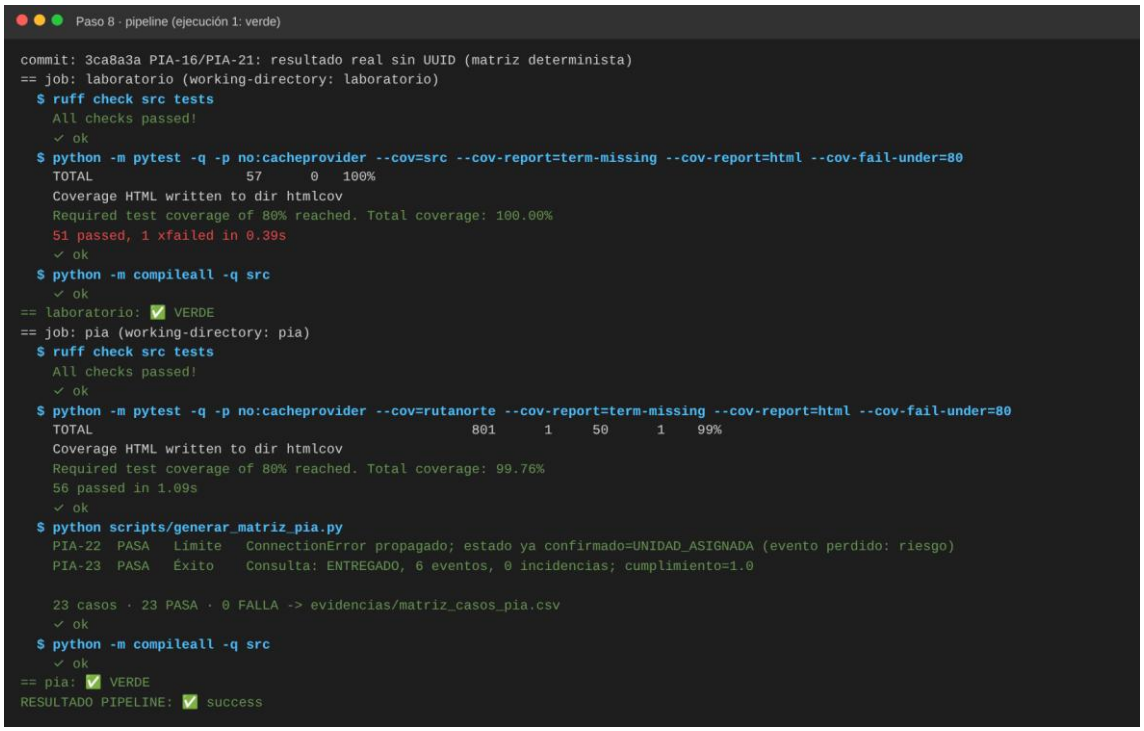


Figura 13. Simulación local del pipeline, ejecución 1: ambos jobs en verde.

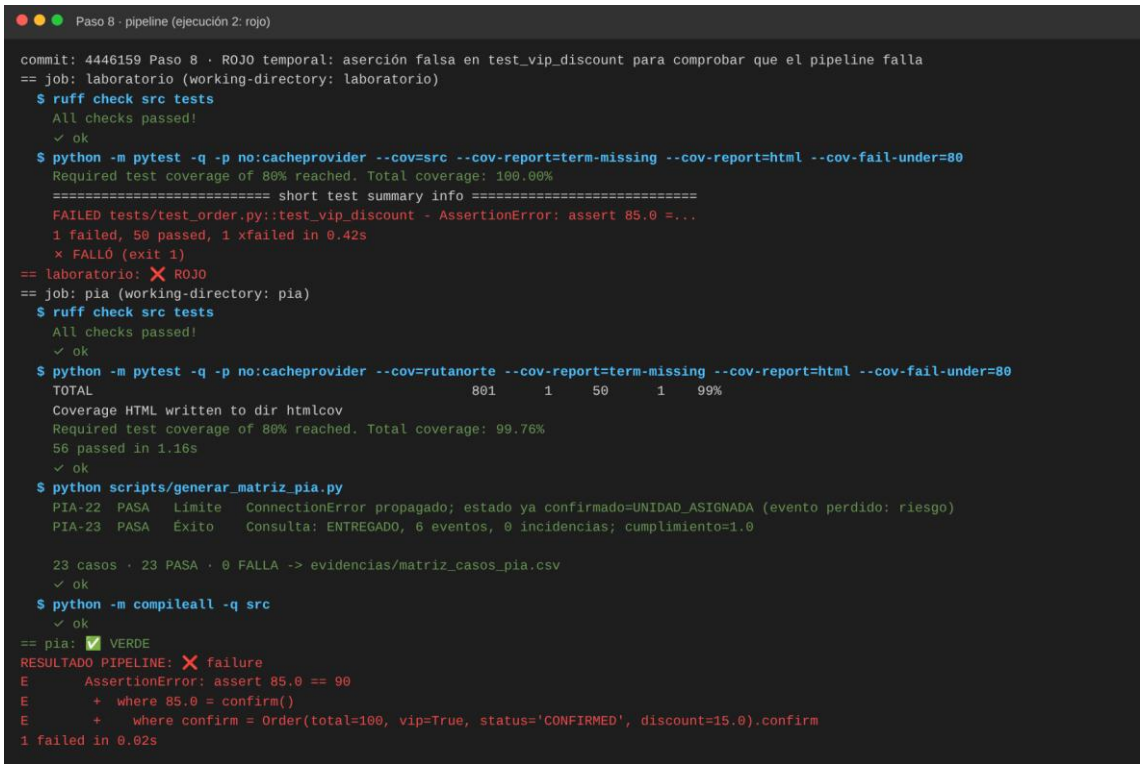


Figura 14. Simulación local, ejecución 2: el job Laboratorio falla en pytest (assert 85.0 == 90).

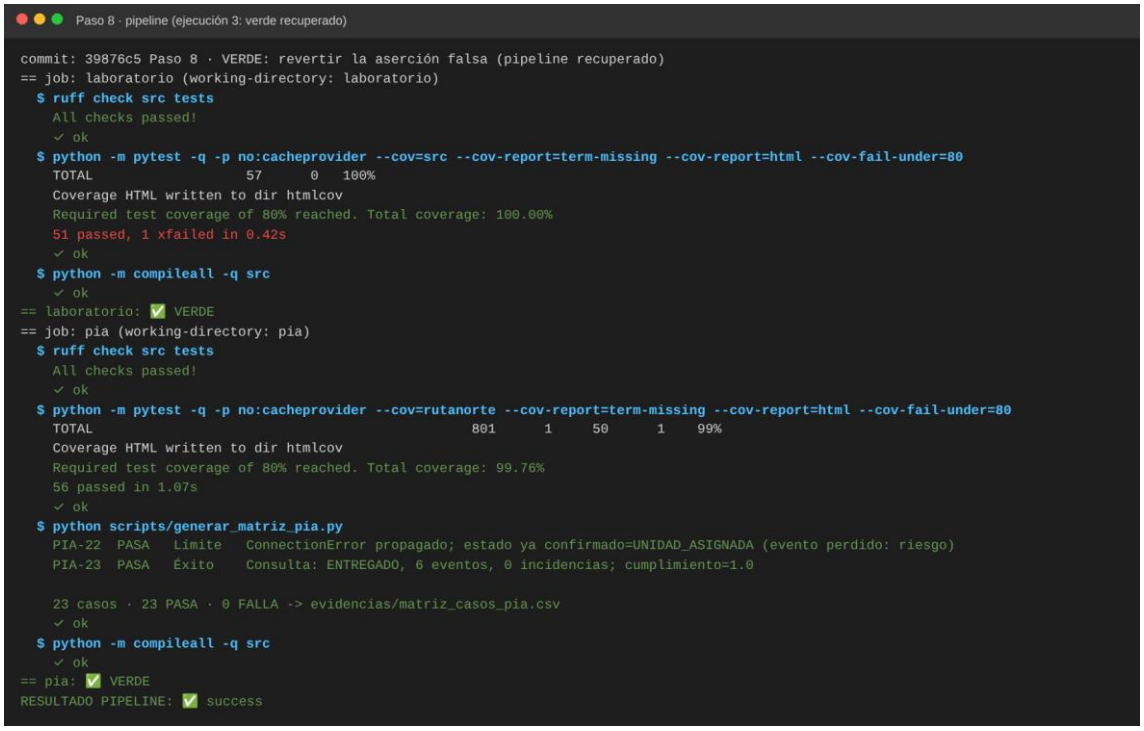


Figura 15. Simulación local, ejecución 3: tras revertir, el pipeline regresa a verde.

¿Qué detectó el pipeline?

En la ejecución roja Ruff pasó (el código es válido), pero pytest detectó que test_vip_discount esperaba 90 cuando el dominio cobra 85; el paso terminó con código 1 y el job quedó en rojo antes de llegar a compileall. El job del PIA siguió en verde porque es independiente, lo que permite ubicar el fallo de inmediato. Al revertir, ambos jobs volvieron a verde. Importante: el YAML solo marca la ejecución; para **impedir el merge** hay que activar branch protection o un ruleset en main que exija los checks “Laboratorio (pedidos)” y “PIA RutaNorte Logistics”.

Ejecución en GitHub Actions

El proyecto se publicó en GitHub (https://github.com/ximenaovallev/CLN_AU5_Equipo2) y el workflow CI Business Logic se ejecutó en la pestaña Actions en cada commit a main. La primera ejecución falló en el job Laboratorio (exit code 2): el workflow llama a pytest sin python -m y el laboratorio no tenía configurado pythonpath, por lo que las pruebas no encontraban el paquete src; en la laptop no se notó porque usamos python -m pytest. Se corrigió agregando pythonpath = ["."] en laboratorio/pyproject.toml. Después se obtuvo la secuencia pedida por la guía: #2 verde (corrección), #3 roja (aserción falsa: test_vip_discount espera 90) y #4 verde (aserción revertida). El job PIA RutaNorte Logistics se mantuvo en verde en todas.

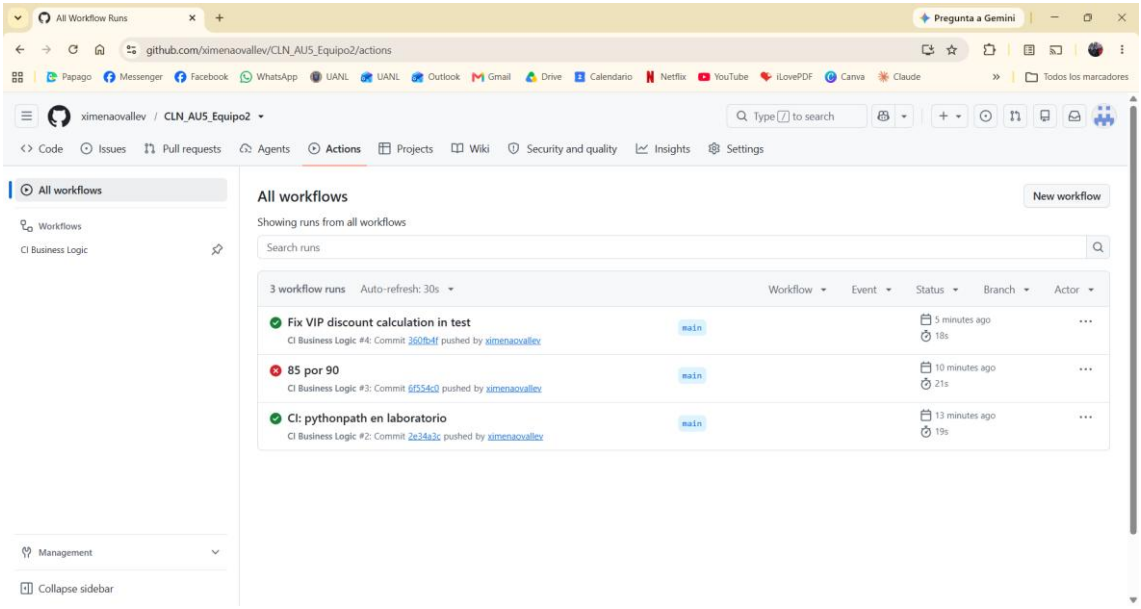


Figura 16. GitHub Actions: ejecuciones #2 (verde), #3 (roja) y #4 (verde) del workflow CI Business Logic.

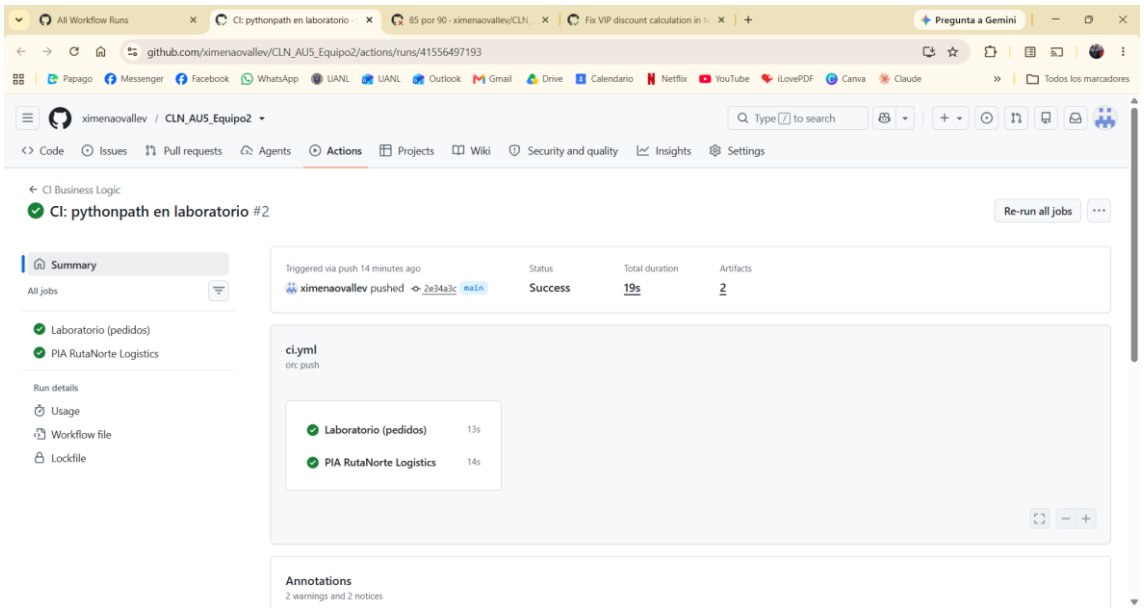


Figura 17. GitHub Actions #2: Laboratorio (pedidos) y PIA RutaNorte Logistics en verde.

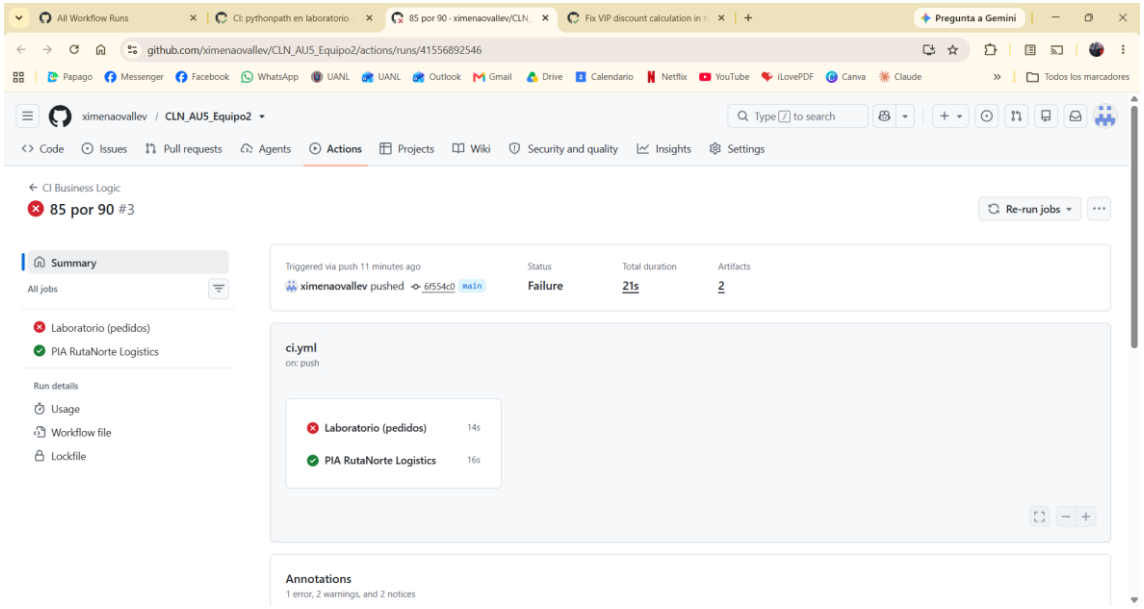


Figura 18. GitHub Actions #3: el job Laboratorio falla y el job PIA sigue en verde.

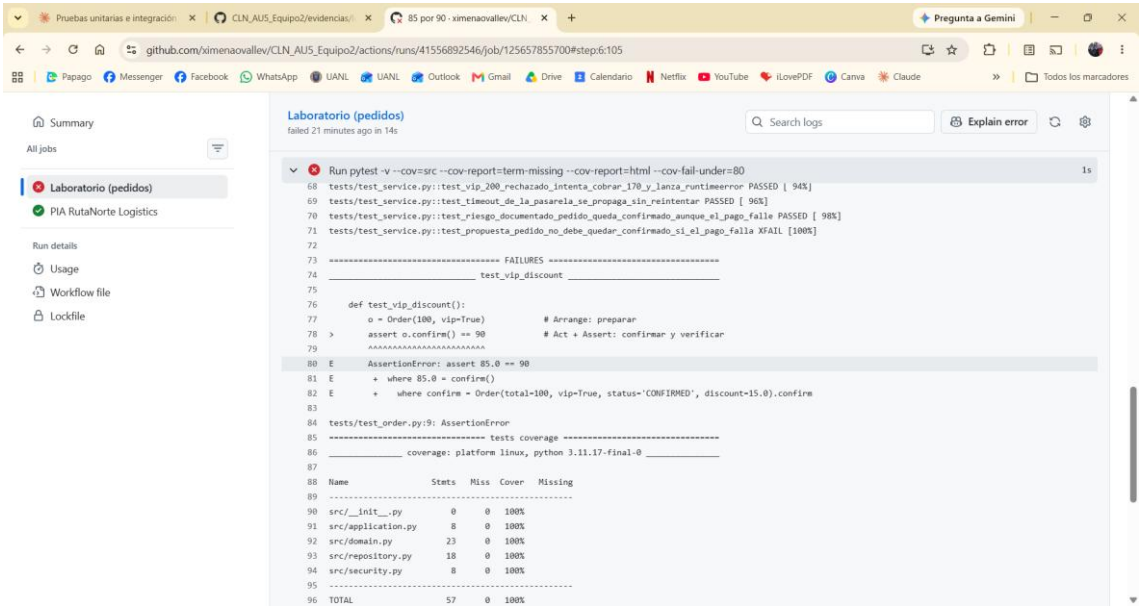


Figura 19. GitHub Actions #3, paso pytest: AssertionError: assert 85.0 == 90 en test_vip_discount.

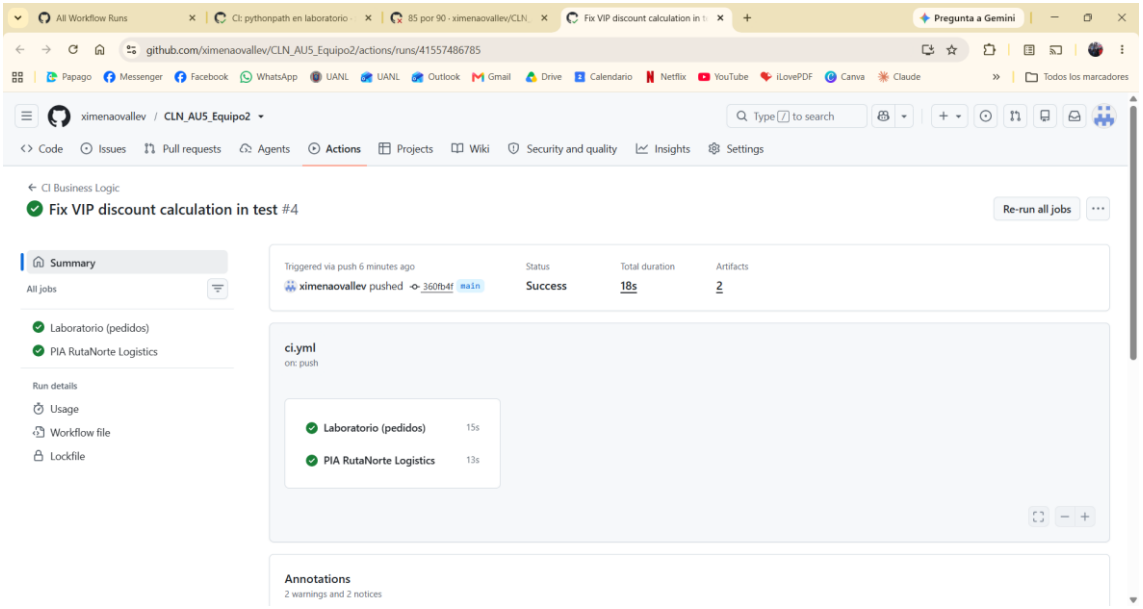


Figura 20. GitHub Actions #4: tras revertir la aserción, ambos jobs regresan a verde.

URL del repositorio: https://github.com/ximenaovallev/CLN_AU5_Equipo2/actions

11. E9 · Paso 9: aplicación a RutaNorte Logistics (PIA)

Se usaron las entidades, agregados, servicios, repositorios y políticas reales de la Unidad 3 (OrdenDeServicio, Cotizacion, AsignadorDeRecursos, PoliticaDeCapacidad, AutoridadDeCancelacion, IUnitOfWork, IPublicadorDeEventos). El código de la U3 se importó sin cambios de comportamiento (solo formato de Ruff) y sus 30 pruebas siguen pasando. No se copiaron las reglas de pedidos.

11.1 Reglas seleccionadas

Regla	Origen en el PIA	Casos (éxito / error / límite)
R1 · La orden de servicio solo se genera tras aceptar la cotización	US-06, T4, CU-02, ADR-001	PIA-01 / PIA-02 / PIA-03
R2 · La unidad debe tener capacidad suficiente y el operador debe estar disponible	US-07, US-08, G2, ADR-004 (S-01 en PIA-08)	PIA-04 / PIA-05 / PIA-06, 07, 08
R3 · La entrega exige llegada a destino, receptor y evidencia; una incidencia no entrega el servicio	US-12, US-13, G4, CU-04, N4	PIA-09 / PIA-10 / PIA-11, 12
R4 · Autoridad de cancelación (seguridad) — supuesto S-04 pendiente de validar	US-05, CU-06, ADR-006	PIA-13 / PIA-14 / PIA-15
Persistencia (commit, rollback, sustitución, eventos)	ADR-002, hallazgo SOLID #5 de la U3	PIA-16 a PIA-19
Servicio externo simulado: bus de eventos	N3, IPublicadorDeEventos	PIA-20 / PIA-21 / PIA-22
Flujo completo CU-01→CU-05	N1, N6, N7, S-05	PIA-23

11.2 Matriz de casos (evidencias/matriz_casos_pia.csv)

La definición vive en pia/tests/casos_pia.csv. Cada prueba registra el resultado real que observó (record_property) y scripts/generar_matriz_pia.py une ambos en la matriz con su estado. Una prueba adicional verifica que cada fila de la matriz tenga una función de prueba ejecutable y que se cumpla el mínimo de la guía (≥9 casos, ≥3 reglas, ≥3 de cada tipo).

ID	Regla	Entrada	Resultado esperado	Tipo	Nivel	Estado
PIA-01	R1 La orden solo se genera tras aceptar la cotización	Cotización COT-1 ACEPTADA; ruta MTY-QRO	Orden en CONFIRMADO, 1 entrada de historial y evento OrdenDeServicioGenerada	Éxito	Unitaria	PASA
PIA-02	R1 La orden solo se genera tras aceptar la cotización	Cotización COT-1 EN_PROCESO	OrdenNoGenerable (no se crea orden)	Error	Unitaria	PASA
PIA-03	R1 La orden solo se genera tras aceptar la cotización	Cotización COT-1 RECHAZADA y luego intento de aceptarla	OrdenNoGenerable y CotizacionNoModificable (estado terminal)	Límite	Unitaria	PASA
PIA-04	R2 Unidad con capacidad suficiente y operador disponible	Carga 800 kg / 4.5 m3; unidad 5000 kg / 30 m3; OP-1 disponible	Asignacion(U-GRANDE; OP-1)	Éxito	Unitaria	PASA
PIA-05	R2 Unidad con capacidad suficiente y operador disponible	Carga 800 kg; unidad 5000 kg; OP-2 NO disponible	RecursosNoDisponibles (operador)	Error	Unitaria	PASA

PIA-06	R2 Unidad con capacidad suficiente y operador disponible	Carga 800 kg / 4.5 m3; unidad EXACTA 800 kg / 4.5 m3	Asignación permitida (igual a la capacidad = suficiente)	Límite	Unitaria	PASA
PIA-07	R2 Unidad con capacidad suficiente y operador disponible	Carga 800.01 kg; unidad 800 kg	RecursosNoDisponibles (excede por 10 g)	Límite	Unitaria	PASA
PIA-08	R2 Unidad con capacidad suficiente	Tope configurable volumen_maximo_m3=4.0; carga 4.5 m3; unidad 30 m3	RecursosNoDisponibles (tope de volumen de la empresa)	Límite	Unitaria	PASA
PIA-09	R3 La entrega exige llegada a destino; receptor y evidencia	Orden LLEGADA_A_DESTINO; destinatario 'Juan Pérez (INE)'; foto de entrega	ENTREGADO, evidencia asociada y evento EntregaRegistrada	Éxito	Integración	PASA
PIA-10	R3 La entrega exige llegada a destino; receptor y evidencia	Orden EN_TRANSITO; destinatario y evidencia	TransicionDeEstadoInvalida; estado EN_TRANSITO sin cambios	Error	Unitaria	PASA
PIA-11	R3 La entrega exige llegada a destino; receptor y evidencia	Orden LLEGADA_A_DESTINO; destinatario=None	Resultado.fallo; estado LLEGADA_A_DESTINO y sin eventos	Límite	Integración	PASA
PIA-12	R3 La entrega exige llegada a destino;	Orden ENTREGADO; registrar incidencia DESTINATARIO_AUSENTE	TransicionDeEstadoInvalida (estado terminal)	Límite	Unitaria	PASA

	receptor y evidencia					
PIA-13	R4 Autoridad de cancelación	Rol Operaciones; orden CONFIRMADO; solicitante Cliente CLI-1; motivo 'Cambio de fecha'	CANCELADO; registra quién y motivo; evento ServicioCancelado	Éxito	Integración	PASA
PIA-14	R4 Autoridad de cancelación	Rol Ventas; orden CONFIRMADO	SinAutoridadParaCancelar; orden sin cambios y sin eventos	Error	Integración	PASA
PIA-15	R4 Autoridad de cancelación	Rol Operaciones (autorizado); orden ENTREGADO	TransicionDeEstadoInvalida: nadie cancela un servicio entregado	Límite	Integración	PASA
PIA-16	Persistencia: commit de la Unit of Work	CU-02 con SqlAlchemyUnitOfWork (SQLite en memoria)	Fila 'orden' visible desde una sesión nueva con estado CONFIRMADO	Éxito	Integración	PASA
PIA-17	Persistencia: rollback de la Unit of Work	CU-03 con OP-2 no disponible sobre SqlAlchemyUnitOfWork	Rollback: fila sigue CONFIRMADO sin asignación y sin eventos	Error	Integración	PASA
PIA-18	Persistencia: sustitución de Liskov	Mismo flujo CU-01..CU-04 sobre EnMemoria y sobre SqlAlchemy	Ambas implementaciones producen el mismo estado final ENTREGADO y la misma secuencia de eventos	Límite	Integración	PASA
PIA-19	Persistencia: eventos no se duplican al recargar	CU-03 y luego CU-06 sobre la misma orden recargada desde SQL	Cada evento se publica una sola vez	Límite	Integración	PASA
PIA-20	Servicio externo: bus de eventos	CU-03 válido; publicador = Mock(spec=IPublicadorDeEventos)	publicar() llamado 1 vez con [RecursosAsignados] después de confirmar	Éxito	Integración	PASA

PIA-21	Servicio externo: bus de eventos	CU-03 con unidad sin capacidad; publicador Mock	publicar() NO se llama	Error	Integración	PASA
PIA-22	Servicio externo: bus de eventos	CU-03 válido; publicador lanza ConnectionError	ConnectionError se propaga y la asignación YA está confirmada (riesgo documentado)	Límite	Integración	PASA
PIA-23	Flujo completo CU-01→CU-05	Aceptar cotización → generar orden → asignar → recolectar → trasladar → llegar → entregar → consultar	Atención al Cliente ve ENTREGADO con eventos; cumplimiento 1.0 (entrega antes de fecha requerida)	Éxito	Flujo	PASA

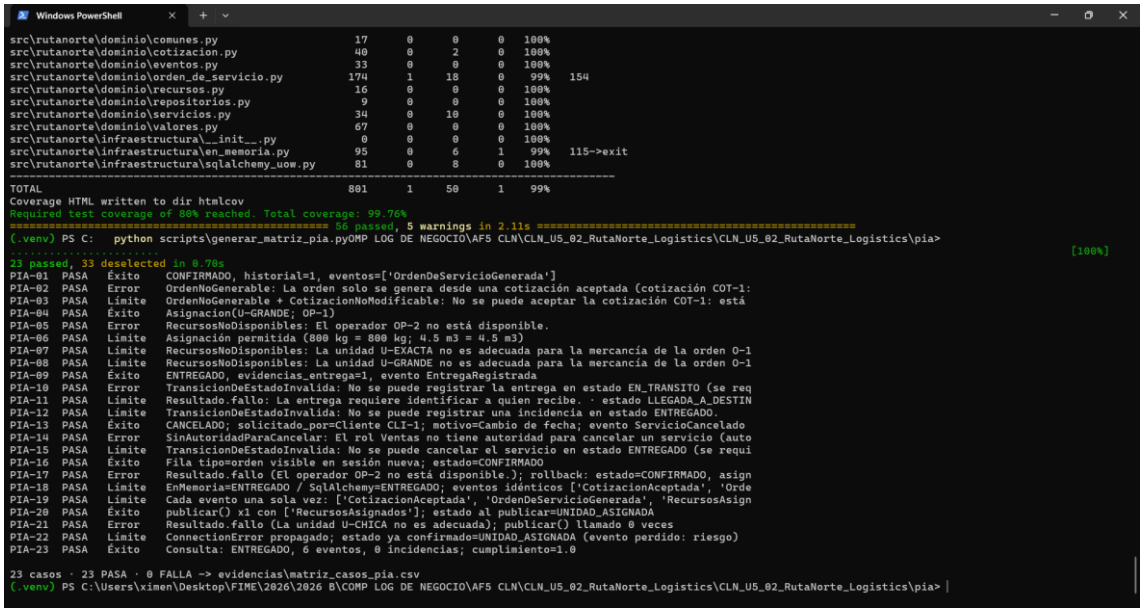


Figura 21. python scripts/generar_matriz_pia.py: 23 casos, 23 PASA, 0 FALLA.

El resultado real completo de cada caso está en la columna resultado_real del CSV (por ejemplo, PIA-17: “Resultado.fallo (El operador OP-2 no está disponible.); rollback: estado=CONFIRMADO, asignacion=None, sin RecursosAsignados”).

11.3 Persistencia: SqlAlchemyUnitOfWork

ADR-002 sigue vigente: el BRD no define la tecnología de base de datos y no la decidimos aquí. Para cumplir “al menos una prueba de persistencia” con commit y rollback reales, se agregó infraestructura/sqlalchemy_uow.py, una **segunda implementación** de IUnitOfWork y de los cuatro repositorios sobre SQLite en memoria, usada solo por las pruebas. Los handlers de la U3 no se modificaron: dependen de interfaces, así que funcionan igual sobre ambas implementaciones (PIA-18). Esto cierra el hallazgo SOLID #5 de la U3 (“la sustitución real no se ha probado”).

¿Cómo demostramos que una prueba detecta un defecto? Se eliminó a propósito la línea del adaptador que evita persistir los eventos de dominio pendientes y PIA-19 falló (los eventos se publicaban dos veces al recargar la orden). Al restaurar la línea, volvió a pasar.

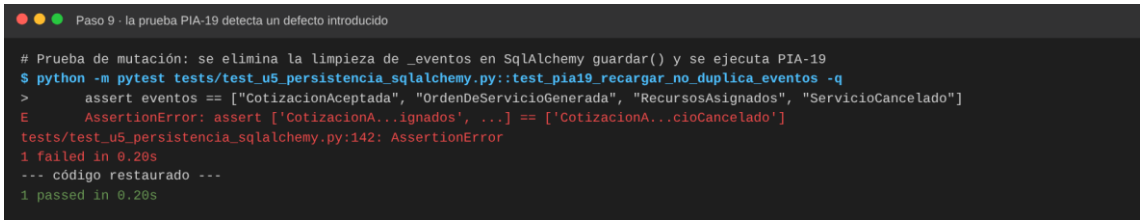


Figura 22. Prueba de mutación: PIA-19 detecta la duplicación de eventos y pasa con el código restaurado.

11.4 Servicio externo: por qué un bus de eventos y no una pasarela

RutaNorte no cobra en línea: el BRD no tiene pasarela de pago, así que no se inventó una. Su integración externa es la notificación de eventos a Operaciones (puerto IPublicadorDeEventos), que responde a N3: dejar de depender de mensajes de WhatsApp. En las pruebas se sustituye por Mock(spec=IPublicadorDeEventos): PIA-20 verifica que se publica una sola vez y **después** de confirmar (el mock revisa el estado de la orden en el momento de la llamada), y PIA-21 que un error de negocio no publica nada.

Riesgo documentado (PIA-22): los handlers confirman y luego publican. Si el bus falla, la excepción sale del handler pero la asignación ya está confirmada y Operaciones no recibe el aviso. Es el mismo tipo de riesgo del Paso 5. Propuesta: patrón Outbox (guardar el evento en la misma transacción y publicarlo después con reintentos).

11.5 Cobertura del código pertinente

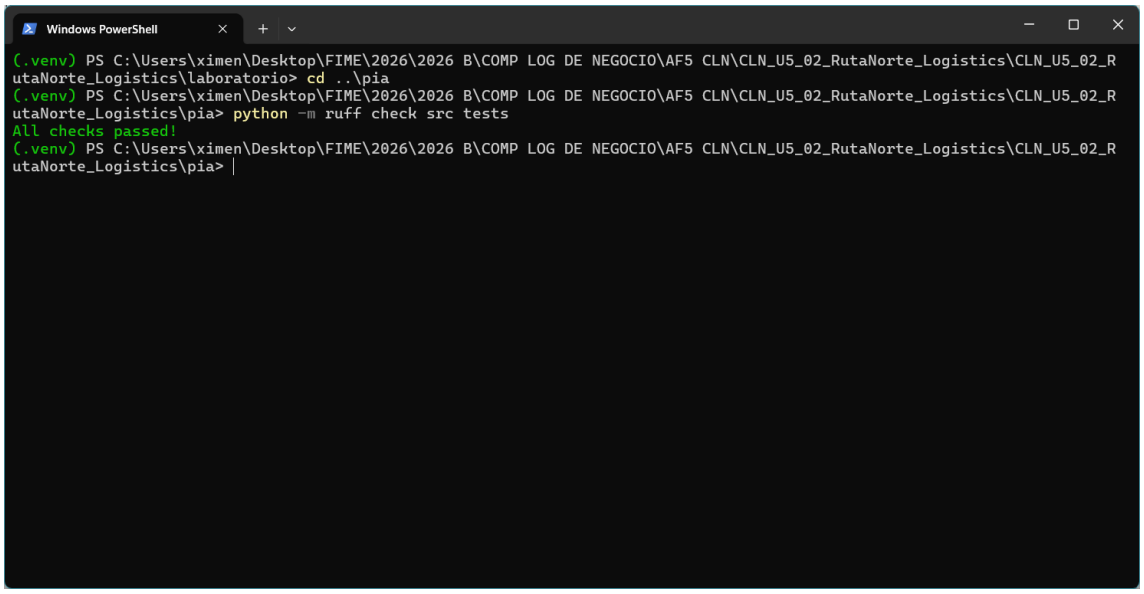


Figura 23. Ruff sin errores en el PIA (python -m ruff check src tests).

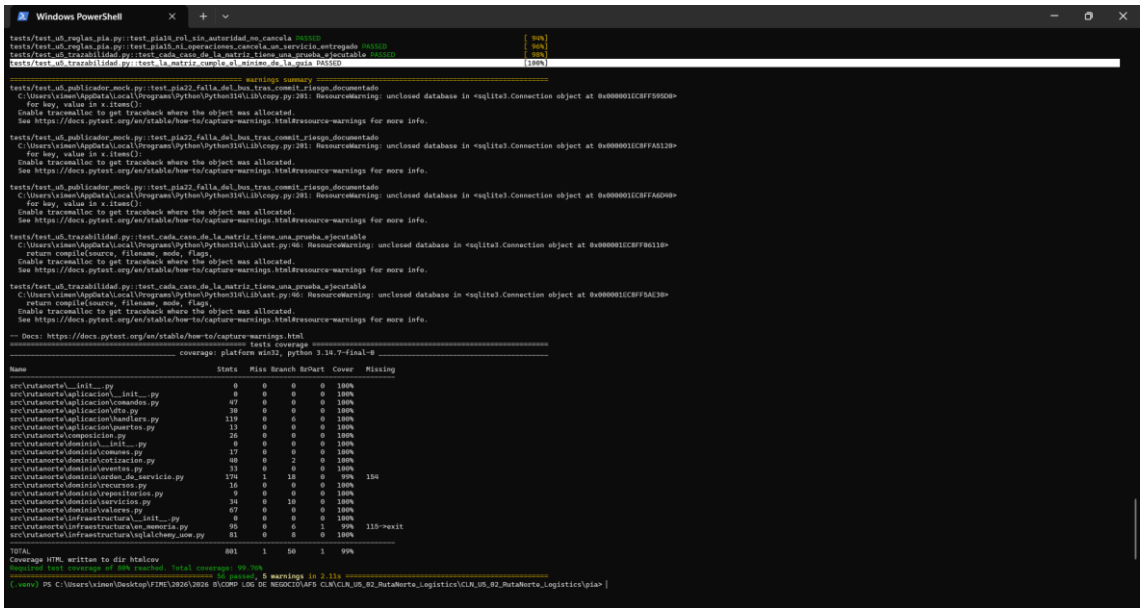


Figura 24. Pruebas del PIA y cobertura de rutanorte: 56 passed, 99.76 %.

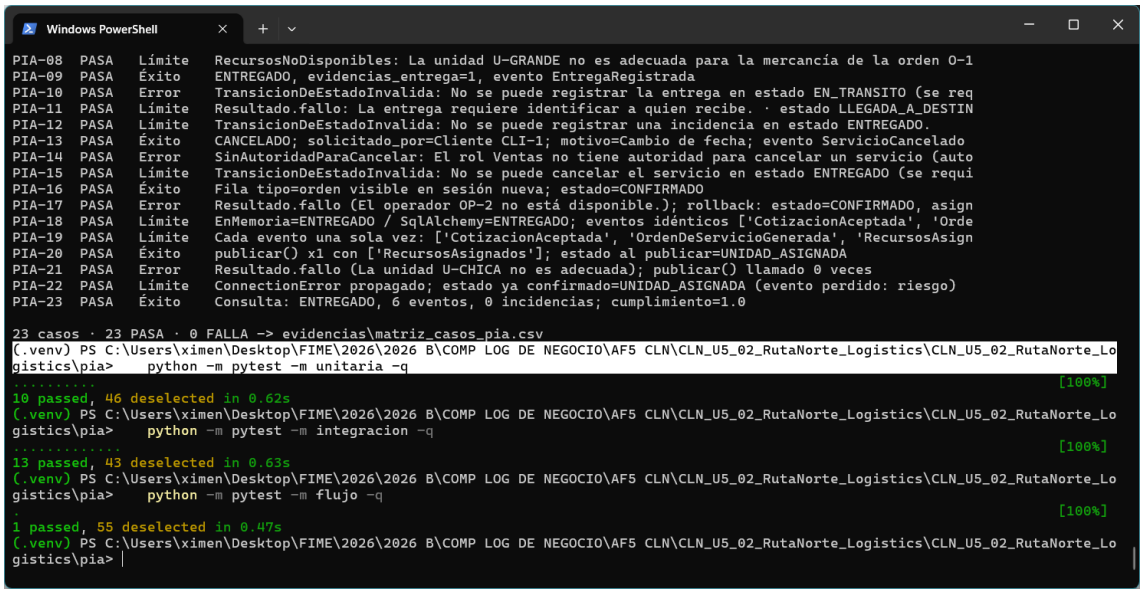


Figura 25. Pruebas U5 por nivel: unitaria 10, integración 13, flujo 1.

Coverage report: 99%

Files | Functions | Classes

filter... hide covered

coverage.py v7.17.0, created at 2026-11-02 13:29 -0600

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src/rutanorte/__init__.py	100%	0	0	0	100%	0	0	100%
src/rutanorte/aplicacion/__init__.py	100%	0	0	0	100%	0	0	100%
src/rutanorte/aplicacion/comandos.py	100%	47	0	0	100%	0	0	100%
src/rutanorte/aplicacion/dto.py	100%	30	0	0	100%	0	0	100%
src/rutanorte/aplicacion/handlers.py	100%	119	0	0	100%	6	0	100%
src/rutanorte/aplicacion/puertos.py	100%	13	0	19	100%	0	0	100%
src/rutanorte/composicion.py	100%	26	0	0	100%	0	0	100%
src/rutanorte/dominio/__init__.py	100%	0	0	0	100%	0	0	100%
src/rutanorte/dominio/comunes.py	100%	17	0	0	100%	0	0	100%
src/rutanorte/dominio/cotizacion.py	100%	40	0	0	100%	2	0	100%
src/rutanorte/dominio/eventos.py	100%	33	0	0	100%	0	0	100%
src/rutanorte/dominio/orden_de_servicio.py	99%	174	1	0	100%	18	0	99%
src/rutanorte/dominio/recursos.py	100%	16	0	0	100%	0	0	100%
src/rutanorte/dominio/repositorios.py	100%	9	0	19	100%	0	0	100%
src/rutanorte/dominio/servicios.py	100%	34	0	4	100%	10	0	100%
src/rutanorte/dominio/valores.py	100%	67	0	0	100%	0	0	100%
src/rutanorte/infraestructura/__init__.py	100%	0	0	0	100%	0	0	100%
src/rutanorte/infraestructura/en_memoria.py	100%	95	0	0	83%	6	1	99%
src/rutanorte/infraestructura/sqlalchemy_uow.py	100%	81	0	0	100%	8	0	100%
Total	**99%**	**801**	**1**	**42**	**98%**	**50**	**1**	**99%**

coverage.py v7.17.0, created at 2026-11-02 13:29 -0600

Figura 26. Reporte HTML de cobertura del PIA.

Las tres ramas que faltaban en la línea base de la U3 (tope de volumen S-01, incidencia sobre un servicio entregado, error de recurso inexistente) se cubrieron con casos de negocio relevantes (PIA-08, PIA-12 y la prueba equivalente sobre SQLAlchemy). Quedan sin ejecutar: la propiedad alto_valor_autorizado (línea 154), que ningún código lee fuera del agregado, y revertir() llamado cuando ya no hay instantánea. No se escribieron pruebas artificiales para ellas; la primera es una pregunta de diseño (¿la necesita algún consumidor?). Los 5 ResourceWarning que aparecen en la laptop tienen la misma causa explicada en E7 (conexiones SQLite en memoria no cerradas explícitamente en Python 3.14) y no afectan los resultados.

```
146         return self._cancelacion
147
148     @property
149     def fecha_entrega(self) -> datetime | None:
150         return self._fecha_entrega
151
152     @property
153     def alto_valor_autorizado(self) -> bool:
154         return self._alto_valor_autorizado
155
156     def evidencias(self, politica: PoliticaDeConservacion | None = None,
157                    hoy: datetime | None = None) -> tuple[Evidencia, ...]:
158         """Todas las evidencias asociadas a la orden; con política (S-03) solo las vigentes."""
159         todas = [*self._evidencias_recepcion, *self._evidencias_entrega,
160                  *(e for i in self._incidencias for e in i.evidencias)]
```

Figura 27. Línea 154 de orden_de_servicio.py sin ejecutar.

11.6 Decisiones de diseño

- **No inventar reglas.** Los casos usan reglas del BRD/historias. Donde la regla depende de un supuesto (S-01, S-04), la prueba lo dice y usa el parámetro con un valor de prueba (por ejemplo, tope 4.0 m³), no una cifra del negocio.
- **Límites exactos.** La regla dice “capacidad suficiente”: 800 kg en una unidad de 800 kg se permite (PIA-06) y 800.01 kg se rechaza (PIA-07). Así se fija la interpretación ≤ en el código.
- **Autorización como prueba de seguridad.** PIA-14 verifica que un rol sin autoridad no cancela y que la orden y los eventos quedan intactos; PIA-15 que ni Operaciones cancela un servicio entregado (análogo a SHIPPED del laboratorio).
- **Observación para el equipo.** En PIA-08 el mensaje de negocio dice “la unidad no es adecuada” aunque el motivo real es el tope de la empresa: el mensaje no distingue la causa. Se propone mejorar el mensaje cuando se valide S-01.

12. Vibe Coding con IA: uso responsable

Se usó un asistente de IA (Claude) para proponer casos de éxito, error y límite, fixtures y mocks a partir del dominio de la U3. Prompt utilizado (el sugerido por la guía, con nuestro contexto):

> Analiza esta clase de dominio y estas reglas aprobadas [OrdenDeServicio, AsignadorDeRecursos,
> AutoridadDeCancelacion; US-06, US-07, US-12, US-13, S-04]. Propón casos de prueba pytest para éxito,
> error y límites. No inventes reglas nuevas. Señala supuestos, riesgos y ramas sin cobertura. Devuelve
> pruebas pequeñas y explica qué verifica cada una.

Propuesta o resultado de la IA	Decisión	Motivo
Probar que al asignar un operador este quede “no disponible” para otras órdenes	Rechazada	El BRD no define esa regla; probarla sería inventarla (restricción del BRD). Queda como pregunta para el stakeholder.
Validar que Peso y Volumen no sean negativos	Rechazada	Ninguna regla aprobada lo exige; se anota como posible invariante a confirmar.
Esperar que el administrador cancele un pedido SHIPPED (como en el slide original)	Corregida	El proyecto implementa que nadie cancela un enviado; se prueba lo implementado (A09) y su análogo en el PIA (PIA-15).
CP08 con resultado esperado CONFIRMED (tal como venía en el CSV)	Corregida	Después del ciclo TDD la expectativa correcta es ValueError: already confirmed.
Registrar el id de la orden en el resultado real de PIA-16 y PIA-21	Corregida	El id es un UUID distinto en cada corrida y hacía la matriz no reproducible.
Adaptador SQLAlchemy que guarda el agregado completo	Aceptada con corrección	La prueba de mutación mostró que persistir los eventos pendientes los duplica al recargar; se limpian antes de guardar (PIA-19).

Todas las pruebas aceptadas se ejecutaron antes de incorporarlas. **La IA propone y el equipo valida**: ninguna prueba se incorporó sin ejecutarla y contrastarla con las reglas aprobadas del PIA.

13. Lista de entrega y autoevaluación

13.1 Lista de entrega (sección 13 de la guía)

Elemento	Qué debe demostrar	Dónde está	Estado
Código src/ y tests/	Pruebas reproducibles y reglas implementadas	laboratorio/ y pia/	Cumplido
Matriz de pruebas CSV	Entradas, expectativas, resultado y estado	evidencias/matriz_resultados_laboratorio.csv, matriz_casos_pia.csv, matriz_autorizacion_E6.csv, tabla_E2_pruebas_unitarias.csv	Cumplido
Cobertura HTML	≥80 % sobre el alcance acordado	htmlcov/ de cada parte (se regenera con pytest; el CI lo publica como artefacto); capturas Figuras 10 y 26	Cumplido (100 % y 99.76 %)
Pipeline CI	Ejecuciones verde, roja y recuperada	.github/workflows/ci.yml; GitHub Actions (https://github.com/ximenaovallev/CLN_AU5_Equipo2); Figuras 13 a 20	Cumplido (#2 verde, #3 roja, #4 verde)
README	Instalación, comandos y arquitectura	README.md	Cumplido
Evidencias E1–E9	Capturas legibles y reflexión de cada paso	Este reporte (evidencias/CLN_AU5_Equipo2_REPORTE.docx y .pdf); evidencias/capturas	Cumplido
Sin .venv, tokens, claves ni datos personales	Entrega limpia	.gitignore; datos ficticios	Cumplido

13.2 Autoevaluación (sección 14 de la guía)

Pregunta	Respuesta
¿Puedo explicar por qué una prueba unitaria no requiere base de datos?	Sí: verifica una regla del dominio (Order, OrdenDeServicio) que no depende de la persistencia; la BD solo añadiría lentitud y causas de fallo ajenas a la regla (E2, sección 2).
¿Tengo evidencia real de RED y GREEN, no solo capturas verdes?	Sí: Figura 3 (RED: DID NOT RAISE ValueError) → Figura 4 (GREEN) → Figura 5 (REFACTOR).
¿Sé diferenciar commit y rollback?	Sí: E4, con lectura desde sesión nueva, rollback que no borra lo confirmado y PIA-16/17.
¿Demostré que el mock no genera un cargo real?	Sí: Mock(spec=PaymentGateway); mock_calls muestra una única llamada simulada; ninguna prueba abre conexiones (E5).
¿Comprobé permisos y límites temporales?	Sí: 24 h exactas denegado, 23:59:59 permitido, admin y SHIPPED; en el PIA, roles de cancelación (E6, PIA-13 a 15).
¿Mi cobertura alcanza 80 % sin ocultar módulos?	Sí: source = src completo y rutanorte completo (incluida infraestructura); no hay omit; los únicos pragma no cover están en código de prueba que nunca se ejecuta.
¿Mi pipeline se ejecutó en GitHub y detectó un fallo?	Sí: en GitHub Actions las ejecuciones #2 (verde), #3 (roja: assert 85.0 == 90) y #4 (verde) lo demuestran (Figuras 16 a 20). Además, la primera ejecución detectó un error real de configuración (pythonpath).
¿Mis casos del PIA representan reglas reales de mi empresa?	Sí: 23 casos de reglas del BRD/historias de RutaNorte; los que dependen de supuestos (S-01, S-04) están marcados como pendientes de validar.

14. Reflexión final

¿Qué defecto de negocio sería más costoso si llegara a producción y qué prueba lo evitaría?

Para RutaNorte Logistics, el defecto más costoso sería que una orden de servicio quede registrada como ENTREGADA sin haber llegado a destino, sin identificar a quien recibe o sin evidencia. La empresa perdería exactamente lo que el BRD busca resolver: ante un reclamo por mercancía faltante o dañada no habría constancia de quién recibió, el indicador de cumplimiento de entregas se inflaría con entregas que no ocurrieron y Atención al Cliente informaría al cliente algo falso. Además, el error sería silencioso: el sistema funcionaría “sin errores” y el daño aparecería semanas después, en una disputa con el cliente o en una decisión tomada con indicadores incorrectos.

Lo evitan las pruebas de la regla R3 trabajadas juntas: PIA-10 rechaza la entrega mientras la orden sigue en tránsito, PIA-11 rechaza la entrega sin receptor sin cambiar el estado ni publicar eventos, y PIA-09 confirma que una entrega válida deja la evidencia asociada a la orden. Como corren en el pipeline en cada push, un cambio que debilite esa regla no podría integrarse sin que la ejecución quede en rojo.

15. Conclusiones

La unidad nos obligó a pasar de “el programa corre” a “tenemos evidencia de que cumple las reglas”. El laboratorio sirvió para practicar cada nivel por separado (unitario, integración con SQLAlchemy, mocks, autorización) y para vivir un ciclo TDD completo con su fase RED documentada (Figuras 3 a 5). Al trasladarlo a RutaNorte, las pruebas se escribieron sobre las reglas que ya habíamos documentado en las Unidades 2 y 3, y respetaron la misma restricción de no inventar políticas: los supuestos se probaron como parámetros.

También aprendimos que la calidad incluye la reproducibilidad: una dependencia sin versión fija hizo fallar el lint del proyecto base sin que el código cambiara, el pipeline de GitHub detectó un error de configuración (pythonpath) que en la laptop no se veía, y un identificador aleatorio hacía que la matriz cambiara en cada corrida. Por último, la cobertura alta no sustituye el criterio: los dos riesgos más importantes que encontramos (confirmar antes de cobrar y publicar eventos después del commit) se detectaron leyendo el flujo y escribiendo pruebas del comportamiento actual, no por el porcentaje.